

Intune PowerShell in Real Work

Practical Zero-to-Advanced SOP for Autopilot, Devices, Apps, Compliance, Configuration, Security, Updates, Reporting, and SCCM

What is different in this edition

This file is organized by actual Intune work. For each topic it explains where the work begins in the portal, whether PowerShell is used locally on the device or remotely through Microsoft Graph, how the script is tested and deployed, where results are checked, how failures are investigated, and how the same topic is explained in an interview.

PowerShell route	Use
Local PowerShell	Inspect or change the Windows endpoint
Intune Platform Script	Deliver a script to assigned Windows devices
Intune Remediation	Detect an issue, run a fix when detected, and report results
Microsoft Graph PowerShell	Read or manage Intune tenant objects such as devices, apps, policies, assignments, and reports
SCCM ConfigurationManager module	Automate ConfigMgr site, devices, collections, applications, content, and update administration

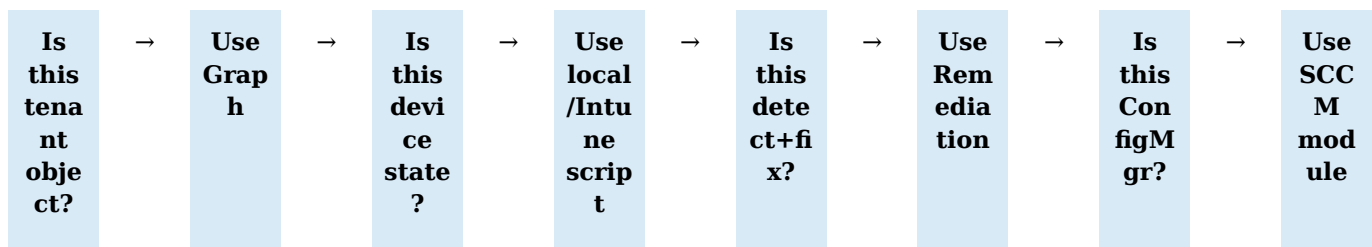
Safe-use rule

Start read-only. Use a lab tenant or pilot group. Use least-privilege permissions. Never test wipe, retire, delete, or broad assignment changes on production. Store scripts in source control, log outcomes, and keep rollback ready.

1. First Understand the Four PowerShell Locations

Location	What runs there	Typical example
Admin workstation	Graph SDK, reports, tenant inventory	Export all noncompliant devices
Intune service	Stores/assigns script or remediation package	Assign detection and remediation to pilot group
Managed Windows device	PowerShell runs through IME as SYSTEM or user	Check service, certificate, registry, app version
SCCM console/site context	ConfigurationManager cmdlets run against site	Audit collections and applications

Core decision



Example

Creating a compliance policy is a tenant operation, so Graph or the portal is used. Checking whether BitLocker is active on the device is a local PowerShell operation. Fixing a repeated local issue is a Remediation use case.

2. Basic Lab Setup Before Intune Automation

1. Use a nonproduction Windows device or VM.
2. Use Visual Studio Code with the PowerShell extension if approved.
3. Keep Windows PowerShell 5.1 and PowerShell 7 understanding separate.
4. Create a folder structure for Source, Tests, Logs, Output, and Release.
5. Use a test Entra group containing only lab devices.
6. Install Microsoft Graph PowerShell modules only from an approved repository.
7. Never place passwords, client secrets, recovery keys, or access tokens in scripts.

```
$PSVersionTable  
Get-ExecutionPolicy -List  
Get-Command Get-Service  
Get-Help Get-Service -Full  
Get-Service | Select-Object -First 5 Name,Status
```

3. Autopilot Record and Profile Assignment

What this topic means

Autopilot has a registration record, group tag, deployment profile assignment, Entra device identity, and managed-device record. These are related but separate objects.

Where the work starts in Intune

8. Import/confirm hardware registration.
9. Set Group Tag through the approved process.
10. Confirm dynamic device group membership.
11. Assign deployment profile and ESP.
12. Validate Assigned status before OOBE.

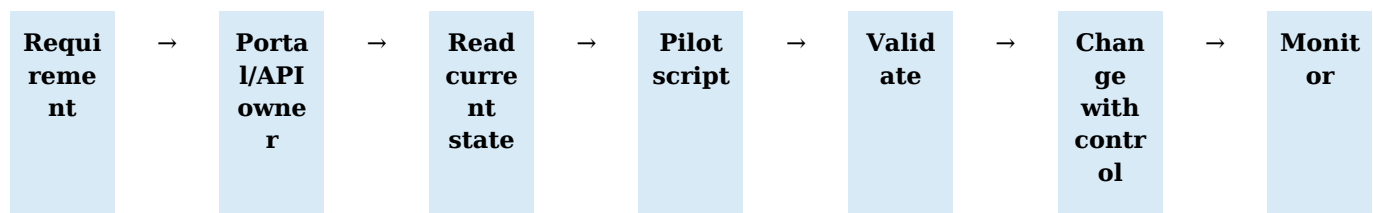
Where PowerShell is actually used

- Use Graph to inventory Autopilot registrations and assignment state.
- Use PowerShell locally to collect hardware information through the approved Autopilot process.
- Use reports to compare serial, group tag, and profile assignment.

Which PowerShell method to choose

Need	Best method
Read tenant Autopilot objects	Microsoft Graph PowerShell
Collect local hardware evidence	Local PowerShell
Deploy setup scripts after enrollment	Intune Platform Script or app

Process flow



Practical PowerShell example

```

Import-Module Microsoft.Graph.Authentication
Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All'
$Context = Get-MgContext
$Context | Select-Object TenantId,Account,Scopes
# Continue only after confirming the expected tenant and read scope.

# Query the documented Autopilot resource only after confirming current API documentation.
# Export serial, group tag, profile assignment, and linked identifiers for review.
  
```

Real work case

Scenario

A laptop reaches normal consumer OOBЕ because its Autopilot profile is not assigned. The admin uses a read-only report to verify registration, group tag, dynamic-group membership, and deployment profile assignment before resetting.

Validation and troubleshooting

- Do not reset first; collect diagnostics.
- Check tenant, serial, profile status, group rule, and synchronization.
- Correlate Autopilot, Entra, and managed-device IDs carefully.

Interview-ready explanation

PowerShell is mainly used for inventory, registration support, bulk reporting, and validation. The actual deployment profile is created in the portal or Graph and is assigned through groups.

4. Autopilot Hardware Hash and Device Information

What this topic means

A hardware hash represents device hardware registration data used in Autopilot onboarding. Collection must follow organizational and Microsoft-supported processes.

Where the work starts in Intune

13. Open approved elevated session on source device.
14. Collect hardware information using the approved script/tool.
15. Securely import through tenant process.
16. Apply group tag and validate profile assignment.

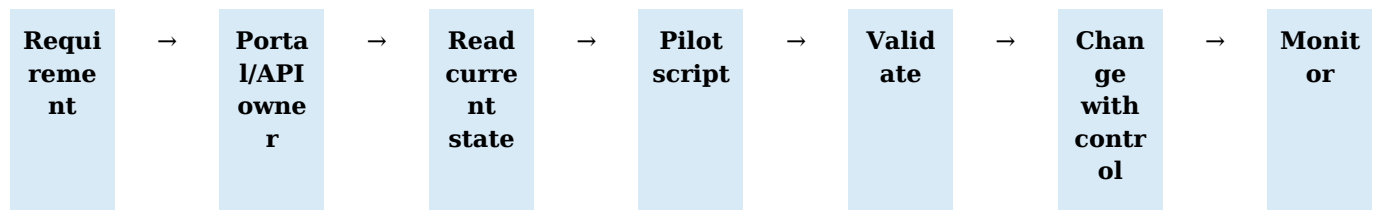
Where PowerShell is actually used

- Run approved local collection script.
- Validate serial and output file.
- Automate naming and secure file placement without exposing device data.

Which PowerShell method to choose

Need	Best method
Single device collection	Local elevated PowerShell
OEM/bulk registration	OEM/partner process
Import and status reporting	Graph or portal

Process flow



Practical PowerShell example

```
# Basic device evidence before approved hash collection
$BIOS = Get-CimInstance Win32_BIOS
$CS = Get-CimInstance Win32_ComputerSystemProduct
[PSCustomObject]@{
  ComputerName = $env:COMPUTERNAME
  SerialNumber = $BIOS.SerialNumber
  UUID = $CS.UUID
  CollectedUtc = (Get-Date).ToUniversalTime()
}
```

Real work case

Scenario

A vendor sends a device with the wrong serial mapping. The admin compares BIOS serial, asset record, registration entry, and OEM information before import.

Validation and troubleshooting

- Run only approved collection tooling.
- Protect CSV/device identifiers.
- Confirm elevated context and internet/proxy requirements.

Interview-ready explanation

I use PowerShell to collect validated device identity evidence and support the registration workflow, but I do not treat a homemade hash routine as a replacement for the supported Autopilot process.

5. Autopilot ESP Troubleshooting

What this topic means

Enrollment Status Page tracks required setup phases and selected apps/policies during enrollment. PowerShell is used to collect evidence, not to bypass ESP.

Where the work starts in Intune

17. Confirm ESP profile and assignments.
18. Identify whether failure is device or account setup.
19. Identify blocking app or policy.
20. Collect enrollment and IME diagnostics.
21. Fix package, detection, network, or assignment, then retest.

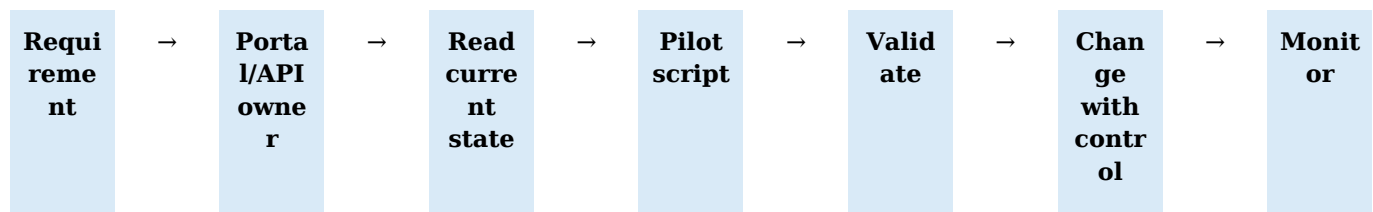
Where PowerShell is actually used

- Use local commands and logs to collect join, MDM, service, and installer evidence.
- Use Graph/portal to review assigned apps and device status.

Which PowerShell method to choose

Need	Best method
Local enrollment evidence	Local PowerShell
Assignment inventory	Graph read report
Repeated local health issue	Remediation after enrollment

Process flow



Practical PowerShell example

```

$Snapshot = [ordered]@{
    TimestampUtc = (Get-Date).ToUniversalTime()
    Computer = $env:COMPUTERNAME
    PowerShell = $PSVersionTable.PSVersion.ToString()
    IMEStatus = (Get-Service IntuneManagementExtension -ErrorAction SilentlyContinue).Status
    JoinSummary = (dsregcmd /status | Select-String 'AzureAdJoined|DomainJoined|DeviceId') -join ';'
}
[PSCustomObject]$Snapshot | ConvertTo-Json -Depth 4
  
```

Real work case

Scenario

ESP fails on a Win32 application. The admin correlates the failing app, IME log, installer log, install

context, network download, return code, and detection rule.

Validation and troubleshooting

- Do not remove all required apps without analysis.
- Check device time, proxy, service endpoints, and join state.
- Compare a working and failing device.

Interview-ready explanation

I trace ESP from profile targeting to device registration, MDM enrollment, IME processing, app install, detection, and reporting.

6. Managed Device Inventory

What this topic means

Managed devices are the main Intune inventory objects containing platform, OS, owner, compliance, user, and last-sync information.

Where the work starts in Intune

22. Open Devices > All devices for a quick view.
23. Define exact report fields and privacy rules.
24. Use Graph read permission.
25. Retrieve all pages, normalize objects, export securely.

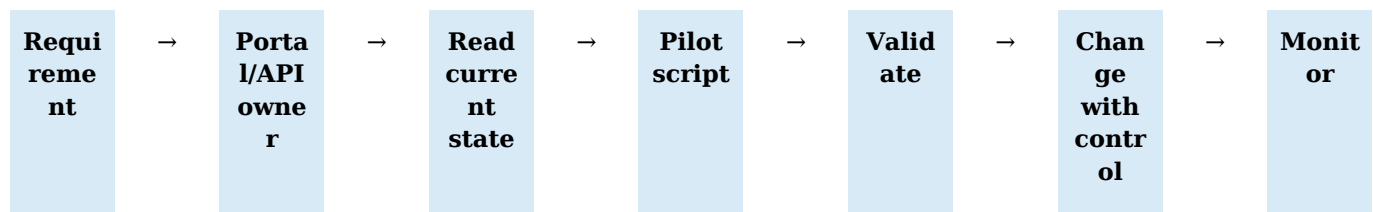
Where PowerShell is actually used

- Create daily or weekly inventory.
- Join with asset tracker carefully.
- Find missing OS, stale check-in, or ownership anomalies.

Which PowerShell method to choose

Need	Best method
Portal ad hoc view	Intune portal
Repeatable report	Graph PowerShell
Local detailed hardware data	Local PowerShell/CIM

Process flow



Practical PowerShell example

```

Import-Module Microsoft.Graph.Authentication
Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All'
$Context = Get-MgContext
$Context | Select-Object TenantId,Account,Scopes
# Continue only after confirming the expected tenant and read scope.
function Invoke-GraphPagedGet {
    [CmdletBinding()]
    param([Parameter(Mandatory)][string]$Uri)
    $All = [System.Collections.Generic.List[object]]::new()
    do {
        $Response = Invoke-MgGraphRequest -Method GET -Uri $Uri
        foreach($Item in @($Response.value)){ $All.Add($Item) }
        $Uri = $Response.'@odata.nextLink'
    } while($Uri)
    return $All
}

```

```
$Uri='https://graph.microsoft.com/v1.0/deviceManagement/managedDevices?
$select=deviceName,operatingSystem,osVersion,complianceState,lastSyncDateTime,managedDeviceOwnerType,userPrincipalName'
$Devices=Invoke-GraphPagedGet $Uri
$Devices | Export-Csv .\ManagedDevices.csv -NoTypeInfo -Encoding UTF8
```

Real work case

Scenario

Asset management requests a current list of Windows devices, OS versions, owners, users, compliance, and last-sync date. The script produces a read-only snapshot with UTC collection time.

Validation and troubleshooting

- Always handle pagination.
- Confirm tenant and scopes.
- Do not email unrestricted user/device data.
- Validate record count against portal/report context.

Interview-ready explanation

Graph PowerShell is used because the task is tenant reporting. Local PowerShell is used only when more endpoint-level hardware evidence is needed.

7. Stale Device Review

What this topic means

A stale device has not checked in within an approved period. Stale does not automatically mean safe to delete.

Where the work starts in Intune

26. Agree the threshold with security, asset, HR, and support owners.
27. Create read-only report.
28. Enrich with owner, user, serial, enrollment, and asset status.
29. Review exceptions.
30. Take lifecycle action only through approval.

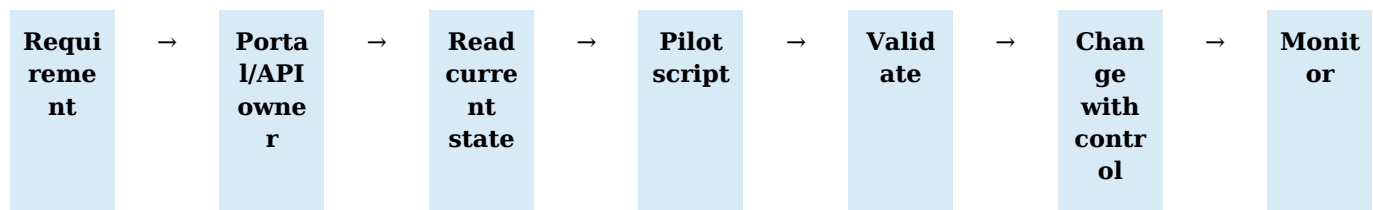
Where PowerShell is actually used

- Calculate age from lastSyncDateTime.
- Produce proposed-action report and no automatic deletion.

Which PowerShell method to choose

Need	Best method
Discovery	Graph read report
Lifecycle decision	Human/change workflow
Approved action	Portal or tightly controlled write automation

Process flow



Practical PowerShell example

```

$Cutoff=(Get-Date).ToUniversalTime().AddDays(-30)
$Stale=$Devices | Where-Object {
    $_.lastSyncDateTime -and ([datetime]$_._lastSyncDateTime -lt $Cutoff)
} | Select-Object
deviceName,operatingSystem,complianceState,lastSyncDateTime,managedDeviceOwnerType,userPrincipalName
$Stale | Export-Csv .\ProposedStaleReview.csv -NoTypeInfo
  
```

Real work case

Scenario

An offboarded user device and an executive travel device are both older than the threshold. The report supports different decisions rather than blindly deleting both.

Validation and troubleshooting

- Check sync timestamps in UTC.
- Check duplicate records and replacement devices.
- Separate retire, delete, lost-device, and asset-return processes.

Interview-ready explanation

I use PowerShell to automate discovery and evidence. I never equate stale with approved deletion.

8. Device Ownership and Primary User

What this topic means

Ownership and primary-user relationships affect policy, reporting, Company Portal, and support experience.

Where the work starts in Intune

31. Report devices marked personal/corporate.
32. Highlight corporate assets marked personal.
33. Review primary user anomalies.
34. Validate serial, asset owner, enrollment method, and HR/support record.
35. Apply approved correction.

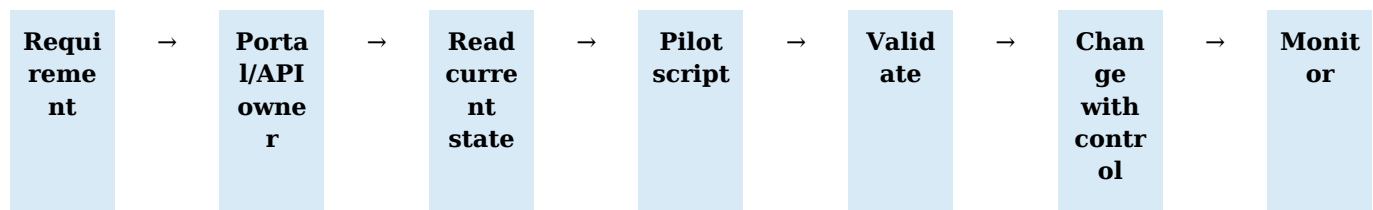
Where PowerShell is actually used

- Graph reports ownership and user relationships.
- Write changes require additional permissions and strict review.

Which PowerShell method to choose

Need	Best method
Audit	Read-only Graph
Correction	Portal or one-device approved write
Shared-device design	Device-targeted policies, no assumed user

Process flow



Practical PowerShell example

```

$Devices | Group-Object managedDeviceOwnerType |
Select-Object @{n='Ownership';e={$_.Name}},Count
$Devices | Where-Object { -not $_.userPrincipalName } |
Select-Object deviceName,operatingSystem,managedDeviceOwnerType,lastSyncDateTime
  
```

Real work case

Scenario

A shared kiosk device has an old primary user. The admin confirms shared-device design before changing the relationship.

Validation and troubleshooting

- Do not assume enrolledBy equals the current user.
- Check shared, kiosk, DEM, and userless scenarios.

Interview-ready explanation

PowerShell helps find anomalies at scale; the correction is controlled because it changes user-device relationships and support behavior.

9. Device Sync and Remote Action Design

What this topic means

Sync asks a device to check in. Other actions such as restart, retire, wipe, Fresh Start, and delete have different risk.

Where the work starts in Intune

36. Confirm the exact device ID and user impact.
37. Start with portal for a single troubleshooting case.
38. Use write automation only with approved use case, exact targeting, audit, and confirmation.
39. Monitor subsequent device status.

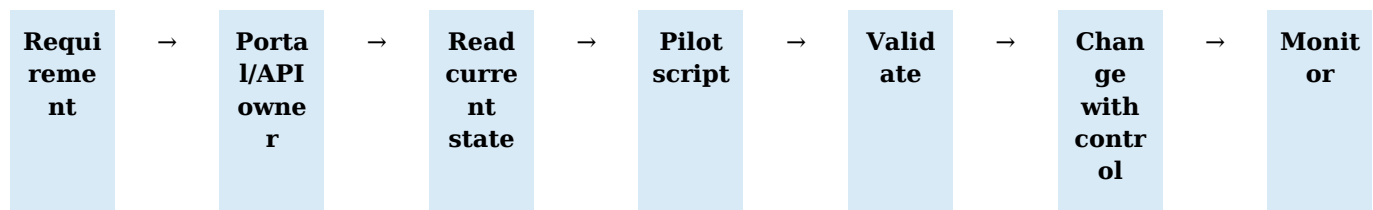
Where PowerShell is actually used

- Use Graph to request actions only after documented permissions and safeguards.
- Use read-only reporting to verify target list before write.

Which PowerShell method to choose

Need	Best method
Single low-risk action	Portal usually preferred
Repeatable approved action	Controlled Graph function with ShouldProcess
Destructive action	Two-person approval and strict operational process

Process flow



Practical PowerShell example

```

function Invoke-ApprovedDeviceAction {
    [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')]
    param([Parameter(Mandatory)][string]$ManagedDeviceId)
    if($PSCmdlet.ShouldProcess($ManagedDeviceId, 'Approved remote action')){
        throw 'Insert only the reviewed, documented Graph action after approval.'
    }
}
Invoke-ApprovedDeviceAction -ManagedDeviceId 'DEVICE-ID' -WhatIf
  
```

Real work case

Scenario

A technician asks for a bulk wipe script. The correct response is to produce a reviewed target report,

require approval, and avoid an unguarded bulk command.

Validation and troubleshooting

- Validate immutable device ID, not only display name.
- Check action support and device online state.
- Record requester, reason, ticket, and result.

Interview-ready explanation

I distinguish low-risk sync from destructive actions and explain safety gates before syntax.

10. Compliance Policy Creation and Inventory

What this topic means

Compliance policies evaluate device state. PowerShell manages tenant objects through Graph and can report policy, assignment, status, and actions. Local PowerShell can validate the underlying endpoint state.

Where the work starts in Intune

40. Start in Devices > Compliance > Policies.
41. Define platform, requirements, actions, grace, and notification.
42. Create in portal for learning and export/inspect structure.
43. Use Graph only after schema and permissions are verified.
44. Assign to pilot and monitor before Conditional Access.

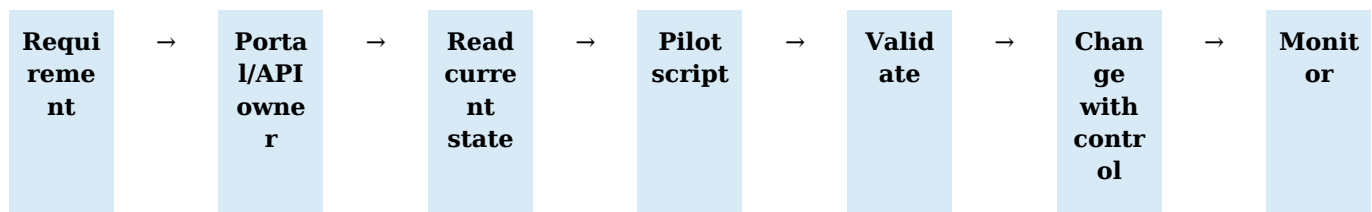
Where PowerShell is actually used

- Graph reads policy inventory and assignments.
- Graph can create/update policy with a fully validated resource payload.
- Local PowerShell checks BitLocker, Defender, firewall, OS, or custom-compliance evidence.

Which PowerShell method to choose

Need	Best method
Learn/create first policy	Portal
Repeatable standardized policy	Graph with tested JSON and change control
Validate device state	Local PowerShell
Report compliance	Graph/report export

Process flow



Practical PowerShell example

```
# Read-only managed-device compliance summary
$Uri='https://graph.microsoft.com/v1.0/deviceManagement/managedDevices?
$select=deviceName,complianceState,lastSyncDateTime,operatingSystem'
$Rows=Invoke-GraphPagedGet $Uri
$Rows | Group-Object complianceState |
Select-Object @ {n='State';e={$_.Name}},Count
```

Real work case

Scenario

BitLocker requirement shows noncompliance. The admin checks the compliance result, encryption policy assignment, local protection status, key escrow, and last sync.

Validation and troubleshooting

- Do not confuse configuration with compliance.
- Use the official resource schema for the target platform.
- Start Conditional Access in report-only.

Interview-ready explanation

For policy creation, PowerShell does not get pasted inside the compliance policy. Graph PowerShell creates or reports the policy object; local scripts validate device state; custom compliance is a separate script-plus-JSON scenario.

11. Custom Compliance Script and JSON

What this topic means

Custom compliance uses a discovery script on the device and a JSON rules file that defines expected values and remediation messages.

Where the work starts in Intune

45. Define a measurable device condition.
46. Write PowerShell discovery returning only expected JSON.
47. Create matching JSON rule with data type and operator.
48. Upload script, add policy custom setting, assign pilot.
49. Monitor errors and device setting results.

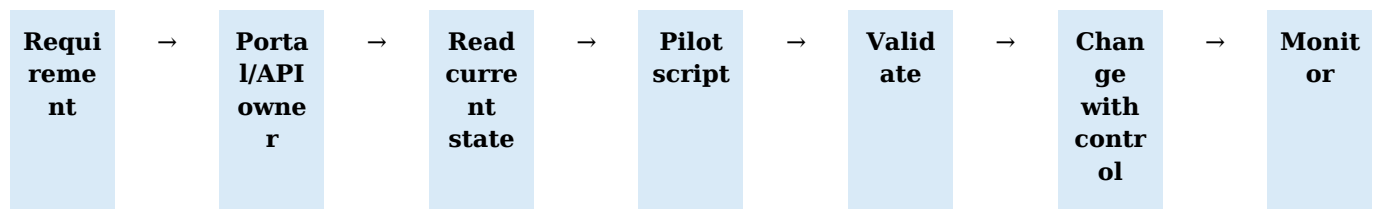
Where PowerShell is actually used

- PowerShell discovers local state.
- JSON defines the compliance rule; Graph is optional for tenant automation.

Which PowerShell method to choose

Need	Best method
Discovery logic	PowerShell on device
Expected rule	JSON uploaded to policy
Policy creation/assignment	Portal or Graph

Process flow



Practical PowerShell example

```

$Result=[ordered]@{
    CompanyServiceRunning = ((Get-Service -Name 'CompanyService' -ErrorAction SilentlyContinue).Status -eq 'Running')
    CompanyRegistryValue = (Get-ItemPropertyValue -Path 'HKLM:\SOFTWARE\Company' -Name 'Configured' -ErrorAction SilentlyContinue)
}
$Result | ConvertTo-Json -Compress
  
```

Real work case

Scenario

A custom check fails because the script writes informational text before JSON. The admin removes extra output and aligns JSON SettingName and data types.

Validation and troubleshooting

- Return valid JSON only.
- Test under intended execution context.
- Match Boolean/integer/string types exactly.
- Keep user remediation accurate.

Interview-ready explanation

This is one of the places where PowerShell is actually embedded into the Intune policy workflow: the script discovers state, and JSON tells Intune what is compliant.

12. Configuration Profile Creation and Reporting

What this topic means

Configuration profiles apply settings through Settings Catalog, templates, ADMX, or Custom CSP. PowerShell can report or create tenant policy objects through Graph, while local PowerShell validates effective state.

Where the work starts in Intune

50. Start Devices > Configuration > Create.
51. Choose the correct profile type.
52. Build small pilot profile in portal to learn settings and scope.
53. Use Graph export/report to document settings and assignments.
54. Create through Graph only after testing exact payload.

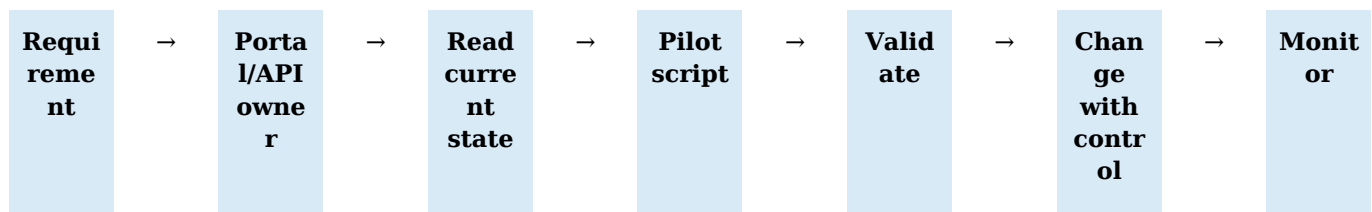
Where PowerShell is actually used

- Graph reads policies, settings, assignments, and status.
- Graph can create policy with resource-specific JSON.
- Local PowerShell validates registry, CSP-related evidence, services, files, and app policy pages.

Which PowerShell method to choose

Need	Best method
Settings discovery and first build	Portal
Standardized repeatable object	Graph with source-controlled JSON
Endpoint verification	Local PowerShell
Conflict audit	Graph plus per-setting report

Process flow



Practical PowerShell example

```
# Safe policy inventory pattern
Connect-MgGraph -Scopes 'DeviceManagementConfiguration.Read.All'
# Use the currently documented Graph resource for the specific policy type.
# Normalize: Id, Name, Platform, Technology, Created, Modified, Assignments.
Get-MgContext | Select-Object TenantId,Account,Scopes
```

Real work case

Scenario

Edge policy shows conflict. Graph inventory finds legacy and Settings Catalog policies; local validation checks edge://policy and relevant device evidence.

Validation and troubleshooting

- Settings Catalog schemas are nested and resource-specific.
- Do not invent JSON from a screenshot. Export/test a known-good pilot.
- Check user versus device scope.

Interview-ready explanation

PowerShell has two roles: tenant automation through Graph and endpoint validation locally. A script is not automatically inserted into a normal Settings Catalog setting.

13. Custom OMA-URI and PowerShell Validation

What this topic means

Custom OMA-URI configures a documented Windows CSP when no native catalog/template setting meets the requirement.

Where the work starts in Intune

55. Confirm official CSP path, scope, data type, values, editions, and rollback.
56. Create Custom profile in portal or Graph.
57. Assign lab device.
58. Use MDM event logs and local checks to validate.
59. Retire Custom profile when native setting becomes available.

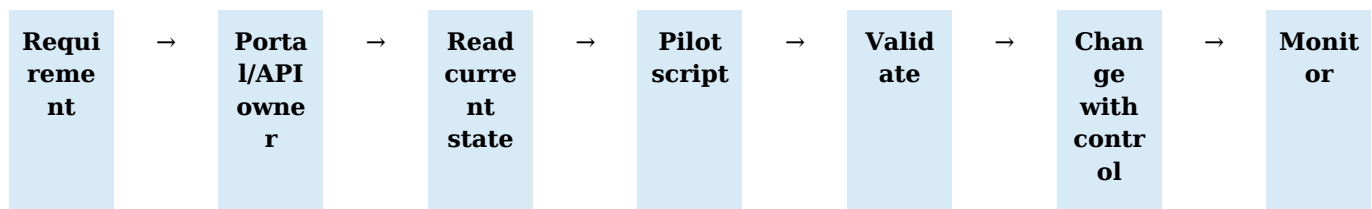
Where PowerShell is actually used

- Graph can create/report the Custom profile.
- Local PowerShell can query logs, registry, files, services, or MDM diagnostics depending on the CSP.

Which PowerShell method to choose

Need	Best method
CSP configuration	Portal/Graph
Device verification	Local PowerShell and event logs
Repeated health issue	Remediation only if not fighting policy

Process flow



Practical PowerShell example

```
Get-WinEvent -LogName 'Microsoft-Windows-DeviceManagement-Enterprise-Diagnostics-Provider/Admin' -MaxEvents 100 |
Select-Object TimeCreated,Id,LevelDisplayName,Message
```

Real work case

Scenario

Custom OMA-URI reports error. The admin checks exact path, device/user scope, data type, Windows support, XML escaping, and DeviceManagement event log.

Validation and troubleshooting

- Do not use registry as a universal proof of CSP state.
- Use official CSP documentation.
- Document inverse/remove behavior.

Interview-ready explanation

PowerShell is used for local evidence and Graph automation around the profile; the OMA-URI itself is a policy payload, not a random PowerShell command.

14. Endpoint Security Policy Automation

What this topic means

Endpoint security includes antivirus, firewall, disk encryption, account protection, ASR, EDR, App Control, and baselines.

Where the work starts in Intune

60. Choose one authoritative management surface.
61. Create focused policy in Endpoint security.
62. Pilot Audit or low-impact settings first.
63. Use Graph for inventory and controlled object automation.
64. Use local PowerShell and Defender/Intune reports for validation.

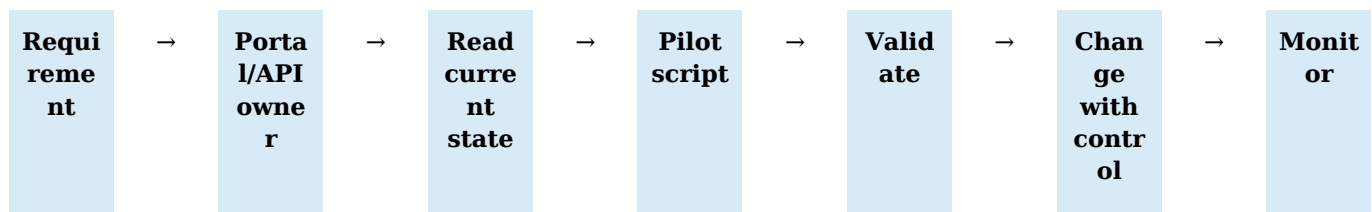
Where PowerShell is actually used

- Graph reports endpoint security policy metadata and assignments.
- Local cmdlets inspect Defender, firewall, BitLocker, services, and events.
- Remediations address approved local support issues, not policy conflicts.

Which PowerShell method to choose

Need	Best method
Policy definition	Portal/Graph
Security telemetry	Intune/Defender reports
Local state	PowerShell cmdlets
Targeted support fix	Remediation

Process flow



Practical PowerShell example

```

$Defender=Get-MpComputerStatus
$Firewall=Get-NetFirewallProfile
$BitLocker=Get-BitLockerVolume -MountPoint $env:SystemDrive
[PSCustomObject]@{
  DefenderRealtime=$Defender.RealTimeProtectionEnabled
  SignatureAge=$Defender.AntivirusSignatureAge
  FirewallProfilesOn=(@($Firewall|Where-Object Enabled).Count)
  BitLockerProtection=$BitLocker.ProtectionStatus
}
  
```

Real work case

Scenario

Security dashboard shows unhealthy device. The admin compares policy assignment, local state, Defender sensor, signatures, firewall profiles, encryption, and reporting freshness.

Validation and troubleshooting

- Do not expose recovery keys.
- Third-party antivirus can affect Defender mode.
- Check baseline/profile conflicts.

Interview-ready explanation

I use Graph for tenant policy inventory and PowerShell locally for objective evidence. Security changes are staged and reviewed with the security team.

15. Defender Antivirus Script Use

What this topic means

Defender PowerShell cmdlets inspect health and can perform approved actions. Intune Antivirus policy remains the configuration authority.

Where the work starts in Intune

65. Create Antivirus policy and assignment in Endpoint security.
66. Use reports to identify unhealthy endpoints.
67. Use local PowerShell to inspect status and update/scan only through approved support procedure.
68. Use Remediation for a repeated safe issue if policy permits.

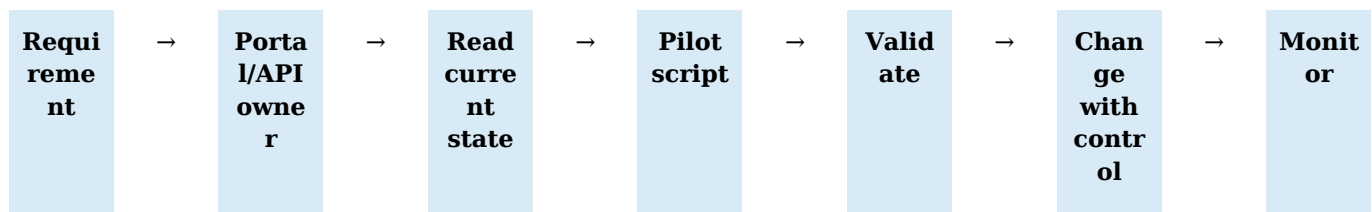
Where PowerShell is actually used

- Get-MpComputerStatus for health.
- Get-MpPreference for local effective preference evidence.
- Update-MpSignature or Start-MpScan only as approved support actions.

Which PowerShell method to choose

Need	Best method
Configuration	Endpoint security policy
Investigation	Local PowerShell and Defender portal
Repeatable fix	Remediation with security approval

Process flow



Practical PowerShell example

```

Get-MpComputerStatus | Select-Object
AMServiceEnabled,AntivirusEnabled,RealTimeProtectionEnabled,AntivirusSignatureAge,QuickScanAge,FullScanAge
Get-MpPreference | Select-Object DisableRealtimeMonitoring,ExclusionPath,ExclusionProcess
  
```

Real work case

Scenario

A device is unhealthy due to old signatures. The admin validates network, Windows Update/Defender channels, service health, third-party antivirus mode, then updates signatures and verifies.

Validation and troubleshooting

- Tamper protection may block local changes.
- Do not add broad exclusions.
- Correlate local and portal time.

Interview-ready explanation

PowerShell is primarily diagnostic and operational; durable Defender configuration belongs in the managed security policy.

16. Firewall Rules and Application Connectivity

What this topic means

Firewall policy and rules are managed centrally. PowerShell validates active local rules and traffic-related attributes.

Where the work starts in Intune

69. Create policy/rule in Endpoint security.
70. Define direction, protocol, ports, addresses, program path, profiles, and interfaces.
71. Assign pilot.
72. Inspect active rule locally and test application.
73. Review logs and status.

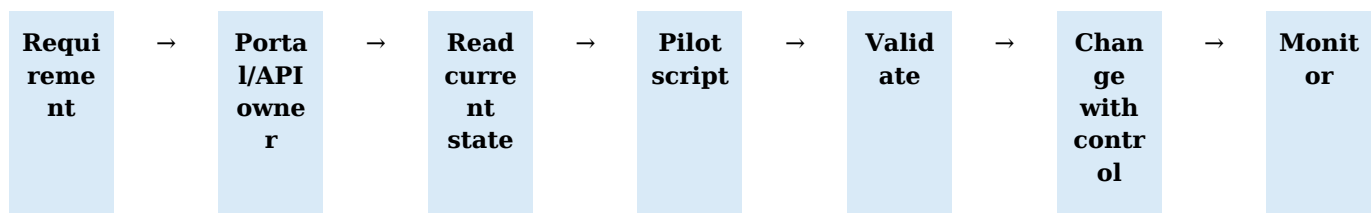
Where PowerShell is actually used

- Get-NetFirewallRule and filter cmdlets inspect effective state.
- Graph reports policy assignments.
- Local scripts can collect evidence but should not create competing permanent rules.

Which PowerShell method to choose

Need	Best method
Permanent rule	Endpoint security policy
Validation	Local PowerShell
Temporary diagnostic	Approved local support process

Process flow



Practical PowerShell example

```

Get-NetFirewallProfile | Select-Object Name,Enabled,DefaultInboundAction,DefaultOutboundAction
Get-NetFirewallRule -PolicyStore ActiveStore |
Where-Object DisplayName -like '*Zscaler*' |
Select-Object DisplayName,Enabled,Direction,Action,Profile
  
```

Real work case

Scenario

Zscaler tunnel fails even though policy succeeded. The admin checks x86/x64 program path, direction, TCP/UDP, ports, remote ranges, profile, service, and active-store rule.

Validation and troubleshooting

- Policy succeeded does not prove app connectivity.
- Check executable path and network profile.
- Avoid broad Any/All rule as a quick fix.

Interview-ready explanation

I validate the exact application flow and active policy rather than adding an unmanaged local allow rule.

17. BitLocker and Encryption Operations

What this topic means

Disk encryption policy enables and configures BitLocker; PowerShell validates readiness, volume status, and protectors without exposing keys.

Where the work starts in Intune

74. Create Disk Encryption policy.
75. Assign pilot and validate TPM/reboot prerequisites.
76. Confirm encryption and recovery-key escrow.
77. Use compliance only after remediation policy works.

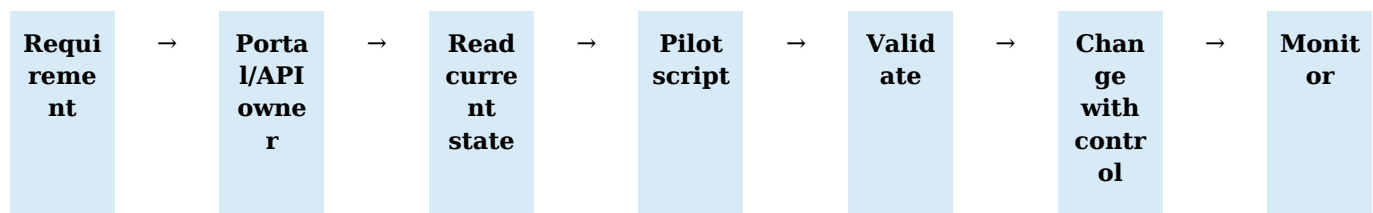
Where PowerShell is actually used

- Get-BitLockerVolume and manage-bde status for local evidence.
- Graph/portal reports managed-device encryption and recovery-key metadata with strict permission.

Which PowerShell method to choose

Need	Best method
Configuration	Endpoint security Disk encryption
Validation	Local PowerShell
Compliance	Compliance policy
Recovery	Approved identity and security process

Process flow



Practical PowerShell example

```
Get-BitLockerVolume -MountPoint $env:SystemDrive |
Select-Object MountPoint,VolumeStatus,ProtectionStatus,EncryptionMethod,EncryptionPercentage
manage-bde -status $env:SystemDrive
```

Real work case

Scenario

Device shows noncompliant for encryption. The admin checks assignment, local volume status, TPM/readiness, required reboot, protection state, and escrow separately.

Validation and troubleshooting

- Never log or export recovery passwords.
- Encryption percentage and protection status are different concepts.

Interview-ready explanation

PowerShell proves local encryption state; Intune policy configures it; compliance evaluates it; recovery follows a controlled process.

18. Windows LAPS and Local Admin Governance

What this topic means

Windows LAPS rotates local administrator passwords and stores recovery information in the approved directory.

Where the work starts in Intune

- 78. Design account and password policy.
- 79. Deploy through Account protection or supported policy.
- 80. Validate policy processing and password backup.
- 81. Use approved retrieval workflow only when required.

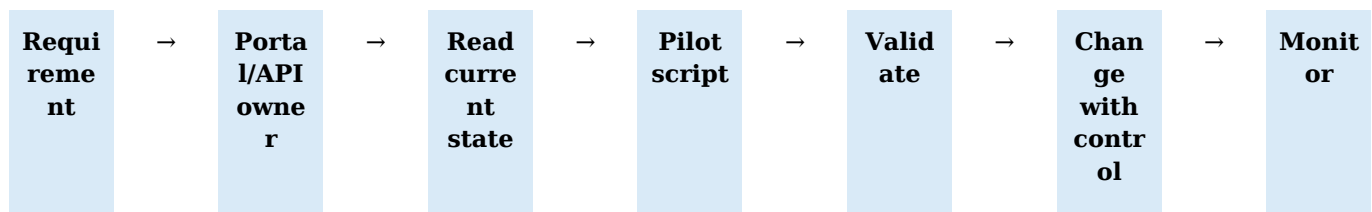
Where PowerShell is actually used

- Local PowerShell can inspect policy/event evidence.
- Graph/Entra retrieval requires privileged permission and auditing; do not automate broad password export.

Which PowerShell method to choose

Need	Best method
Policy	Intune Account protection
Local evidence	PowerShell/event logs
Password retrieval	Privileged portal/API process with audit

Process flow



Practical PowerShell example

```
Get-WinEvent -LogName 'Microsoft-Windows-LAPS/Operational' -MaxEvents 50 -ErrorAction SilentlyContinue |
Select-Object TimeCreated,Id,LevelDisplayName,Message
```

Real work case

Scenario

Help desk cannot retrieve a LAPS password. The admin validates policy, device join, backup directory, permissions, event log, and latest rotation.

Validation and troubleshooting

- Do not export passwords.
- Check RBAC and audit.
- Do not confuse legacy LAPS with Windows LAPS design.

Interview-ready explanation

PowerShell helps troubleshoot processing, but password retrieval is a privileged, audited operation.

19. Attack Surface Reduction Audit to Block

What this topic means

ASR rules reduce risky behaviors. PowerShell and Defender telemetry support audit analysis; durable mode is managed with Endpoint security policy.

Where the work starts in Intune

82. Create ASR policy in Audit.
83. Assign representative pilot.
84. Collect Defender events and business impact.
85. Create narrow approved exclusions.
86. Move selected rules to Warn/Block in waves.

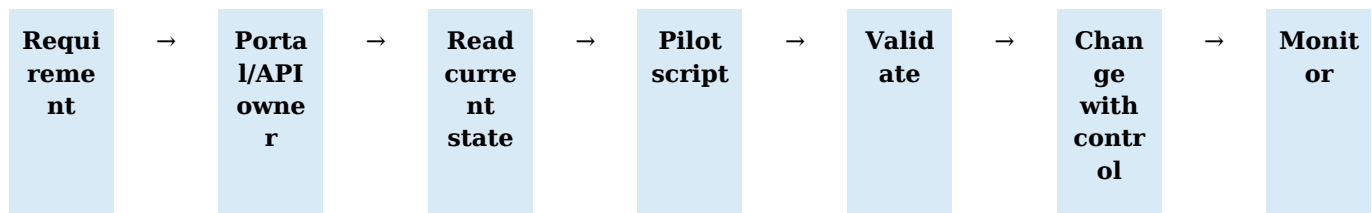
Where PowerShell is actually used

- Get-WinEvent queries Defender Operational events.
- Graph inventories ASR policies and assignments.
- Local PowerShell validates Defender/ASR health.

Which PowerShell method to choose

Need	Best method
Mode and exclusions	Endpoint security policy
Event analysis	PowerShell/Defender Advanced Hunting
Repeated local issue	Do not override policy; fix design

Process flow



Practical PowerShell example

```
Get-WinEvent -LogName 'Microsoft-Windows-Windows Defender/Operational' -MaxEvents 500 |
Where-Object Id -in 1121,1122,5007 |
Select-Object TimeCreated,Id,Message
```

Real work case

Scenario

A finance macro is blocked. The admin identifies rule/event, validates signed application and workflow, creates the narrowest approved exclusion, and retests.

Validation and troubleshooting

- Event IDs and messages vary by scenario; confirm current documentation.
- Do not disable entire ASR policy for one app.

Interview-ready explanation

PowerShell is used to collect evidence from endpoints; the policy lifecycle remains Audit, analyze, pilot, enforce, monitor.

20. EDR Onboarding and Sensor Health

What this topic means

EDR policy onboards devices to Microsoft Defender for Endpoint. PowerShell checks local sensor service and onboarding evidence.

Where the work starts in Intune

87. Configure Intune-Defender connector.
88. Create EDR onboarding policy.
89. Assign pilot.
90. Confirm onboarding in Intune and Defender portal.
91. Investigate missing telemetry.

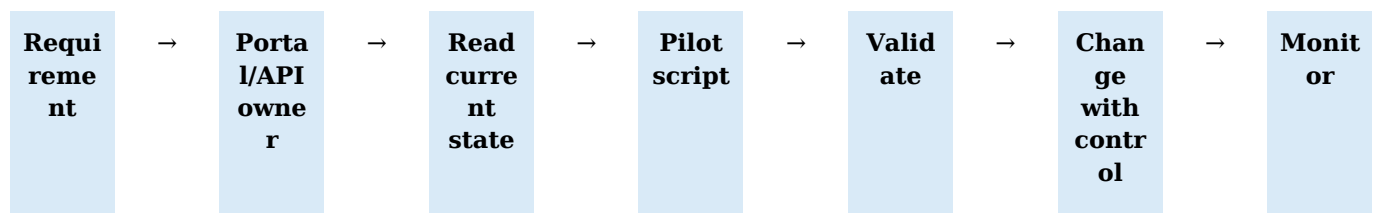
Where PowerShell is actually used

- Local PowerShell checks Sense service and onboarding evidence.
- Graph reports EDR policy metadata and assignments.
- Defender APIs/tools handle security telemetry and incidents.

Which PowerShell method to choose

Need	Best method
Onboarding policy	Endpoint security
Local sensor check	PowerShell
Incident response	Defender portal/security operations

Process flow



Practical PowerShell example

```
Get-Service -Name Sense -ErrorAction SilentlyContinue |
Select-Object Name,Status,StartType
Get-MpComputerStatus | Select-Object AMRunningMode,AntivirusEnabled,RealTimeProtectionEnabled
```

Real work case

Scenario

Intune says policy succeeded but device is missing in Defender. The admin checks connector, licensing, sensor, proxy endpoints, duplicate records, and telemetry time.

Validation and troubleshooting

- Policy success is not onboarding proof.
- Check connector synchronization and device identity.

Interview-ready explanation

PowerShell confirms local sensor health; Defender portal confirms security-service onboarding and telemetry.

21. Win32 Application Packaging and Deployment

What this topic means

PowerShell is widely used for wrappers, prerequisites, installer logging, detection, version checks, uninstall, and remediation.

Where the work starts in Intune

92. Analyze MSI/EXE and vendor switches.
93. Create silent install/uninstall.
94. Add prerequisite checks and logs.
95. Package as .intunewin using supported packaging tool.
96. Create Intune app with detection/requirements/dependencies.
97. Pilot clean install, upgrade, uninstall.

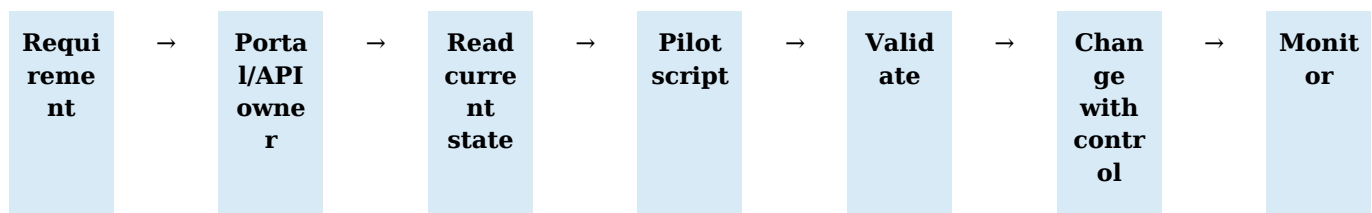
Where PowerShell is actually used

- Local PowerShell wrapper orchestrates installer.
- Detection script determines app state.
- Graph can inventory apps and assignments.

Which PowerShell method to choose

Need	Best method
Install orchestration	PowerShell wrapper
Content packaging	IntuneWinAppUtil
App object/assignment	Portal/Graph
Troubleshooting	IME and installer logs

Process flow



Practical PowerShell example

```

$Installer = Join-Path $PSScriptRoot 'setup.exe'
$Log = "$env:ProgramData\Company\Logs\ExampleApp-install.log"
$Process = Start-Process -FilePath $Installer -ArgumentList '/quiet', '/norestart' -Wait -PassThru
$Process.ExitCode | Out-File $Log -Append
switch($Process.ExitCode){
    0 { exit 0 }
    3010 { exit 3010 }
    default { throw "Installer failed: $($Process.ExitCode)" }
  }

```

```
}
```

Real work case

Scenario

App installs manually but fails through Intune because SYSTEM context cannot access a user path and detection checks HKCU. The admin reworks context and detection.

Validation and troubleshooting

- Use vendor-supported switches only.
- Test SYSTEM context.
- Map documented exit codes.
- Keep detection independent of wrapper success.

Interview-ready explanation

PowerShell wraps and validates installation, but Intune controls delivery, assignment, dependencies, and reporting.

22. Win32 Detection Rules

What this topic means

Detection tells Intune whether the application is installed. It is more important than the installer exit code for final app state.

Where the work starts in Intune

98. Choose MSI, file, registry, or custom script detection.
99. Test absent, present, old version, new version, corrupt, and uninstall states.
100. Test x86/x64 and user/system context.
101. Deploy pilot and review IME logs.

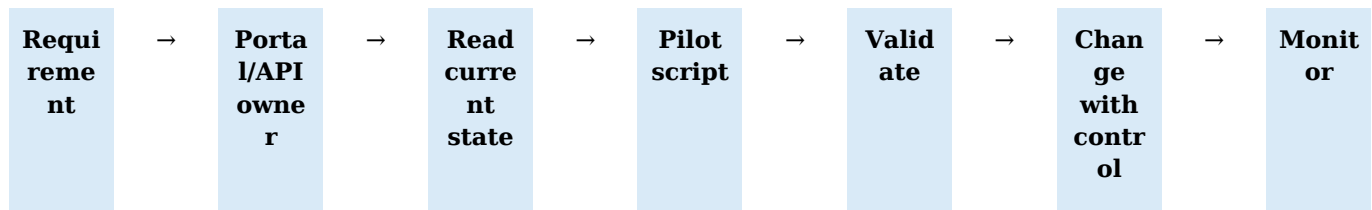
Where PowerShell is actually used

- Custom detection script for complex state.
- Local PowerShell to inspect registry/file/product evidence.

Which PowerShell method to choose

Need	Best method
Simple MSI/file/registry	Built-in detection preferred
Multiple conditions/version logic	Custom PowerShell detection
Per-user app	User-context design

Process flow



Practical PowerShell example

```

$Path='C:\Program Files\Vendor\App\App.exe'
if(Test-Path $Path){
  $Version=[version](Get-Item $Path).VersionInfo.FileVersion
  if($Version -ge [version]'5.2.0'){ Write-Output "Detected $Version"; exit 0 }
}
exit 1
  
```

Real work case

Scenario

Upgrade installs version 5.2 but detection still searches old registry value. The portal reports failure despite successful install.

Validation and troubleshooting

- Custom detection output and exit behavior must match Intune app rules.
- File versions can contain nonstandard strings; test parsing.

Interview-ready explanation

I select the most stable evidence and prove the complete lifecycle, not only the happy path.

23. Application Inventory and Failure Reporting

What this topic means

PowerShell/Graph can inventory apps, owners, versions, assignments, and install statuses for operations and cleanup.

Where the work starts in Intune

102. Define required app fields.
103. Query all pages of mobileApps.
104. Normalize app types.
105. Query assignments/status where supported.
106. Export failure list and correlate IME logs.

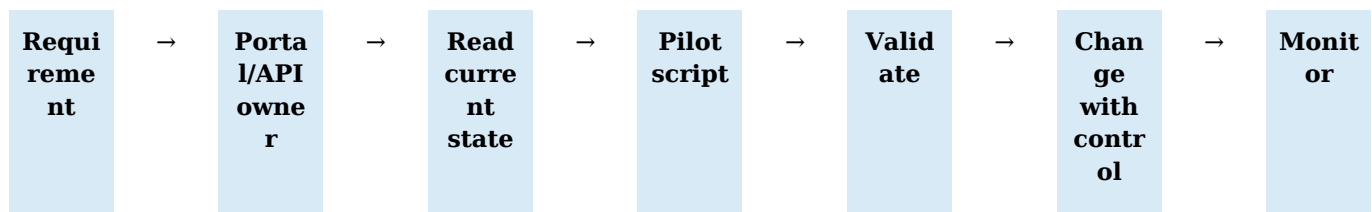
Where PowerShell is actually used

- Graph PowerShell for tenant inventory.
- Local PowerShell/IME logs for root cause.

Which PowerShell method to choose

Need	Best method
App-wide report	Graph
Single-device analysis	IME/local logs
Assignment correction	Portal or controlled Graph write

Process flow



Practical PowerShell example

```

Connect-MgGraph -Scopes 'DeviceManagementApps.Read.All'
# Query the documented mobileApps resource, handle paging,
# then select Id, DisplayName, Publisher, CreatedDateTime, LastModifiedDateTime.
Get-MgContext | Select-Object TenantId,Account,Scopes
  
```

Real work case

Scenario

A new app version produces broad failures. The report identifies affected devices and error codes; rollout is paused through approved change while packaging/detection is fixed.

Validation and troubleshooting

- Different app types have different properties.
- Installation-state endpoints can be resource-specific.

Interview-ready explanation

Graph finds the scope and pattern; local logs explain the installation failure.

24. PowerShell Platform Script Creation

What this topic means

A Platform Script is uploaded into Intune and assigned to Windows users/devices. The PowerShell content runs through IME under configured context.

Where the work starts in Intune

107. Write idempotent script and local tests.
108. Select run as user or SYSTEM.
109. Select script-signature and 64-bit host settings.
110. Upload under Devices > Scripts and remediations > Platform scripts.
111. Assign pilot.
112. Monitor device/user status and IME logs.

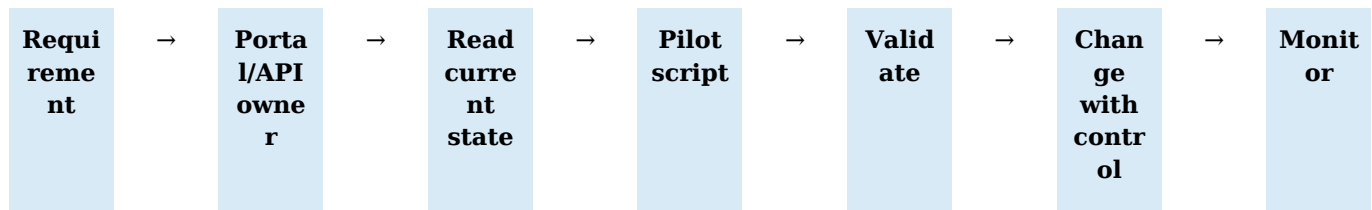
Where PowerShell is actually used

- The .ps1 is the payload.
- Graph may automate script-object creation after exact schema validation.
- Local output/exit behavior supports troubleshooting.

Which PowerShell method to choose

Need	Best method
Simple endpoint configuration/collection	Platform Script
Repeated detect+fix	Remediation
Tenant object management	Graph admin script

Process flow



Practical PowerShell example

```
[CmdletBinding()]
param()
$Log="$env:ProgramData\Company\Logs\PlatformScript.log"
try {
  "$(Get-Date -Format o) Start" | Add-Content $Log
  # Approved, idempotent work here
  "$(Get-Date -Format o) Success" | Add-Content $Log
  exit 0
} catch {
  "$(Get-Date -Format o) ERROR $($_.Exception.Message)" | Add-Content $Log
  exit 1
}
```

Real work case

Scenario

A configuration script works interactively but fails from Intune because it expects a user profile. The admin retests as SYSTEM and removes mapped-drive/HKCU assumptions.

Validation and troubleshooting

- Check IME service and logs.
- Check 32/64-bit selection.
- Check user versus SYSTEM context.
- Check signature requirement and policy conflict.

Interview-ready explanation

Here PowerShell is directly uploaded in Intune. Intune assigns it; IME downloads and runs it; portal status and local logs show the result.

25. Remediation Package Creation

What this topic means

A Remediation package contains detection and optional remediation scripts plus execution metadata and schedule.

Where the work starts in Intune

113. Define one measurable issue.
114. Write fast read-only detection.
115. Write minimal idempotent remediation.
116. Test compliant, detected, remediated, and failed states.
117. Create package under Scripts and remediations.
118. Configure context, signature, schedule, and pilot assignment.
119. Monitor outputs and recurrence.

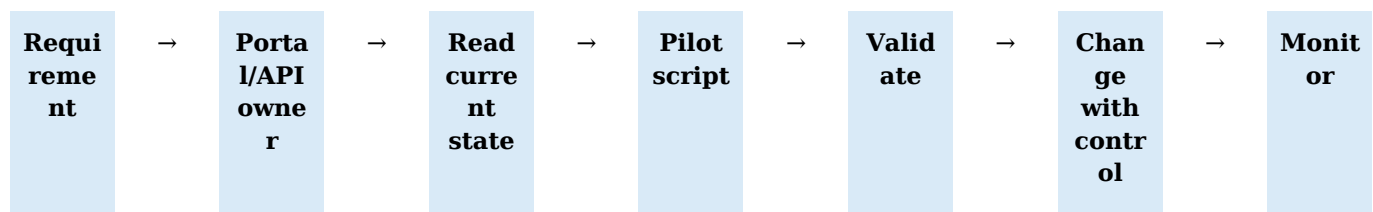
Where PowerShell is actually used

- Detection exit 0 = issue not detected.
- Detection exit 1 = remediation should run.
- Remediation performs fix and exits explicitly.

Which PowerShell method to choose

Need	Best method
Single run configuration	Platform Script
Recurring health check and fix	Remediation
Large tenant report	Graph admin script

Process flow



Practical PowerShell example

```

try {
    $Path = 'HKLM:\SOFTWARE\Company\Endpoint'
    $Value = (Get-ItemProperty -Path $Path -Name Configured -ErrorAction Stop).Configured
    if($Value -eq 1){ Write-Output 'Compliant'; exit 0 }
    Write-Output 'Issue detected'; exit 1
}
catch { Write-Output "Issue detected: $($_.Exception.Message)"; exit 1 }
  
```

Real work case

Scenario

A certificate is near expiry. Detection reports issue; remediation may notify the user or perform only an approved certificate cleanup.

Validation and troubleshooting

- Detection error must not be disguised as compliant.
- Output limits and privacy matter.
- A remediation can recur if root cause remains.

Interview-ready explanation

This is the clearest Intune scripting workflow: create detection and remediation files, upload them as a package, assign a schedule and group, then monitor detection and remediation results.

26. Remediation: Service Health

What this topic means

A recurring service issue can be detected and fixed if the service is organization-owned and policy does not intentionally disable it.

Where the work starts in Intune

120. Define exact service and desired startup/running state.
121. Detect current state.
122. Check dependencies and owner.
123. Remediate only approved state.
124. Verify and report.

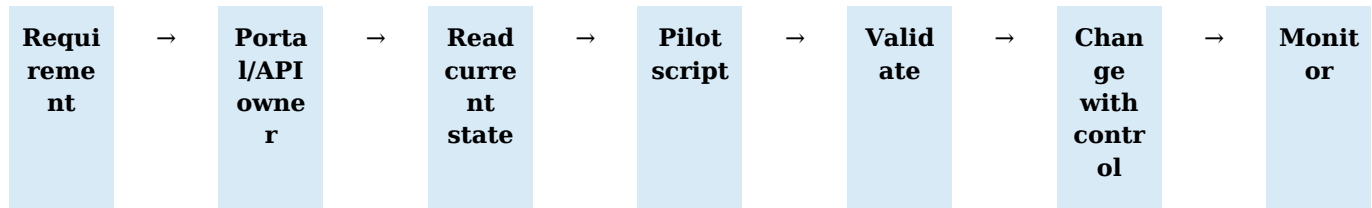
Where PowerShell is actually used

- Get-Service, Get-CimInstance Win32_Service, Set-Service, Start-Service.

Which PowerShell method to choose

Need	Best method
Policy-owned Microsoft security service	Fix policy/design, not local force
Organization app service	Potential remediation after approval

Process flow



Practical PowerShell example

```

$Name='CompanyService'
$Svc=Get-Service $Name -ErrorAction SilentlyContinue
if(-not $Svc -or $Svc.Status -ne 'Running'){ Write-Output 'Issue detected'; exit 1 }
Write-Output 'Healthy'; exit 0
  
```

Real work case

Scenario

Support tickets show a corporate agent service stopped on a small group. A remediation restarts it only after confirming no installer/update is intentionally stopping it.

Validation and troubleshooting

- Check Event Log and service failure reason.

- Do not endlessly restart a crashing service.

Interview-ready explanation

I use remediation only when the desired state is authoritative, safe, and verifiable.

27. Remediation: Disk Space

What this topic means

Low disk blocks updates, apps, logs, and performance. A remediation can report or clean only explicitly approved locations.

Where the work starts in Intune

125. Define threshold and drives.
126. Detect free space.
127. Identify approved disposable content.
128. Calculate potential reclaim.
129. Delete only approved content with logging.
130. Verify free space.

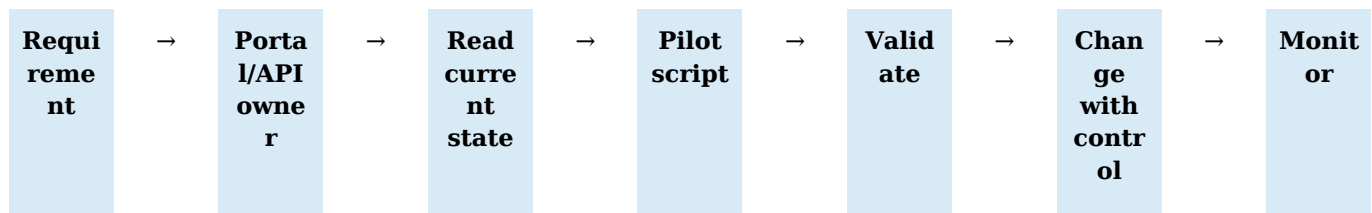
Where PowerShell is actually used

- Get-PSDrive or CIM for free space.
- Remove-Item only for exact allowlisted paths.

Which PowerShell method to choose

Need	Best method
Inventory/report only	Preferred first step
Known app cache cleanup	Remediation after owner approval
User data cleanup	User/support process, not blind script

Process flow



Practical PowerShell example

```

$Drive=Get-CimInstance Win32_LogicalDisk -Filter "DeviceID='C:'"
$FreeGB=[math]::Round($Drive.FreeSpace/1GB,2)
if($FreeGB -lt 10){ Write-Output "Low disk: $FreeGB GB"; exit 1 }
Write-Output "Healthy: $FreeGB GB"; exit 0
  
```

Real work case

Scenario

Feature update fails on low disk. The admin detects affected devices and removes only approved caches; user data remains untouched.

Validation and troubleshooting

- Do not recursively delete temp locations without ownership review.
- Check reserved storage and update requirements.

Interview-ready explanation

PowerShell can detect and quantify low disk; remediation must be explicitly scoped and auditable.

28. Remediation: Certificate Health

What this topic means

Certificates affect Wi-Fi, VPN, S/MIME, and authentication. PowerShell can detect expiry, issuer, subject, and store location.

Where the work starts in Intune

131. Define certificate use and expected issuer/subject.
132. Choose machine or user store.
133. Detect expired/near-expiry certificate.
134. Notify, renew through enrollment, or remove exact stale certificate according to design.

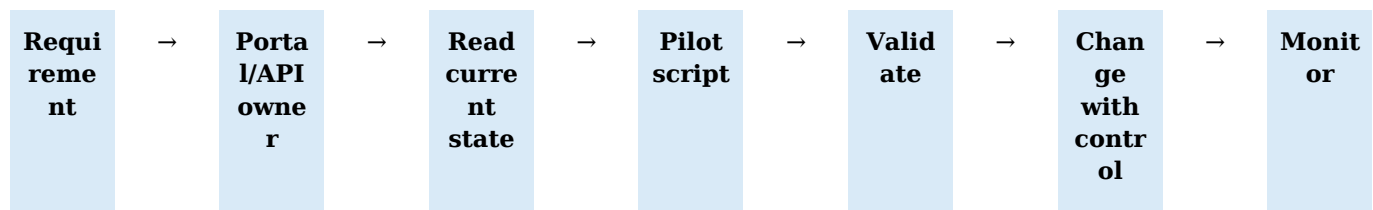
Where PowerShell is actually used

- Cert: provider and X509 properties.
- Do not export private keys or broad-delete certificates.

Which PowerShell method to choose

Need	Best method
Expected renewal	Certificate profile/connector
Health detection	Remediation script
Trust deployment	Trusted certificate profile

Process flow



Practical PowerShell example

```

$Days=30
$Now=Get-Date
$Certs=Get-ChildItem Cert:\LocalMachine\My |
Where-Object { $_.Issuer -like '*Company CA*' -and $_.NotAfter -lt $Now.AddDays($Days) }
if($Certs){ $Certs | Select Subject,Issuer,NotAfter,Thumbprint; exit 1 }
Write-Output 'No matching expiring certificate'; exit 0
  
```

Real work case

Scenario

Corporate Wi-Fi failures begin near certificate expiry. Detection identifies affected devices; certificate enrollment and connector health are fixed rather than deleting all certificates.

Validation and troubleshooting

- Check user versus machine store.
- Check time synchronization and chain.
- Protect thumbprints and identity data in reports.

Interview-ready explanation

PowerShell detects the endpoint symptom; Intune certificate profiles and PKI fix the certificate lifecycle.

29. Update Rings and Patch Reporting

What this topic means

PowerShell reports ring membership, policy settings, last sync, OS build, and update status. Local scripts collect Windows Update evidence.

Where the work starts in Intune

135. Create update rings, feature, quality, and driver policies in Intune.
136. Use Graph to inventory policies and assignments.
137. Use reports to identify errors/noncompliant versions.
138. Use local logs for device-specific root cause.

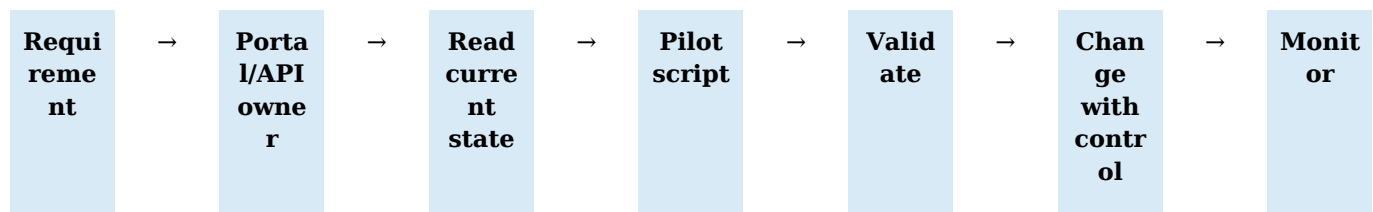
Where PowerShell is actually used

- Graph for tenant policy/reporting.
- Get-WindowsUpdateLog and event logs for local evidence.
- Do not mix incompatible update APIs without design.

Which PowerShell method to choose

Need	Best method
Policy creation	Portal/Graph with validated payload
Status reporting	Intune reports/Graph
Device troubleshooting	Local PowerShell

Process flow



Practical PowerShell example

```
Get-Service wuauserv,usosvc,bits -ErrorAction SilentlyContinue |
Select-Object Name,Status,StartType
Get-WindowsUpdateLog -LogPath "$env:ProgramData\Company\Logs\WindowsUpdate.log"
```

Real work case

Scenario

Pilot devices fail a quality update. The admin checks ring assignment, pause/deferral/deadline, update error, reboot state, storage, and VPN/security agent compatibility.

Validation and troubleshooting

- Generate logs after collecting timestamp/error.
- Do not force production updates outside approved patch process.

Interview-ready explanation

PowerShell supports reporting and diagnostics; Intune policies remain the update authority.

30. Windows Feature Update Readiness

What this topic means

Feature updates depend on hardware, app/driver readiness, disk, safeguard holds, and policy targeting.

Where the work starts in Intune

139. Select feature target version.
140. Assign pilot.
141. Use Intune readiness/reporting.
142. Collect local hardware, disk, BIOS, TPM, app/driver evidence.
143. Resolve blocking issue and retest.

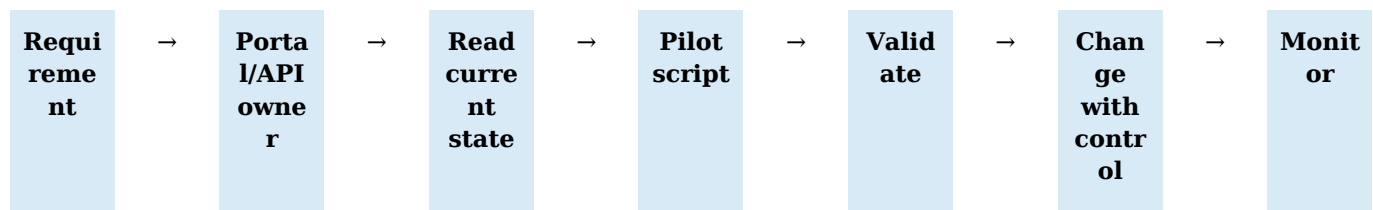
Where PowerShell is actually used

- CIM inventory and local readiness checks.
- Graph reports target policy and device inventory.

Which PowerShell method to choose

Need	Best method
Target version	Feature update policy
Readiness evidence	Reports and local PowerShell
Remediation	App/driver/configuration change

Process flow



Practical PowerShell example

```

$OS=Get-CimInstance Win32_OperatingSystem
$BIOS=Get-CimInstance Win32_BIOS
$Disk=Get-CimInstance Win32_LogicalDisk -Filter "DeviceID='C:'"
[PSCustomObject]@{
    Build=$OS.BuildNumber; BIOSVersion=($BIOS.SMBIOSBIOSVersion -join ',')
    FreeGB=[math]::Round($Disk.FreeSpace/1GB,2)
    TPM=(Get-Tpm -ErrorAction SilentlyContinue).TpmPresent
}
  
```

Real work case

Scenario

A model does not receive Windows 11 feature update. The admin checks target policy, hardware readiness,

TPM, BIOS, safeguards, free space, and driver blocks.

Validation and troubleshooting

- No single script can override a Microsoft safeguard safely.
- Check model-specific vendor guidance.

Interview-ready explanation

PowerShell collects readiness data; Intune controls rollout; safeguard analysis determines compatibility.

31. Policy Assignment Audit

What this topic means

Assignments determine blast radius. PowerShell can document inclusions, exclusions, filters, and broad targets.

Where the work starts in Intune

144. Inventory all policy types.
145. Query assignments and resolve group IDs.
146. Flag All Users/All Devices, no assignment, conflicting scopes, and pilot objects in production.
147. Review with policy owner.
148. Correct through approved change.

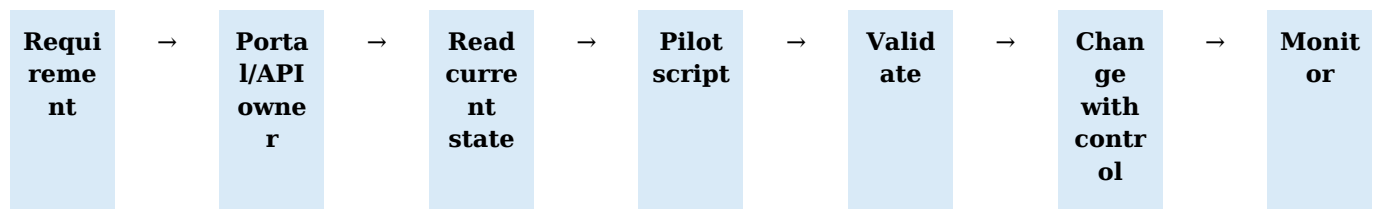
Where PowerShell is actually used

- Graph read inventory first.
- Write assignment automation requires exact schema, group IDs, and ShouldProcess.

Which PowerShell method to choose

Need	Best method
Audit	Graph read
Visual correction	Portal
Standardized deployment pipeline	Graph write with tests and approval

Process flow



Practical PowerShell example

```
# Conceptual output schema
[PSCustomObject]@{
  PolicyName='Example'
  PolicyType='SettingsCatalog'
  IncludedGroups='Pilot Devices'
  ExcludedGroups='Emergency Exclusion'
  Filter='None'
  Risk='Review broad or missing assignment'
}
```

Real work case

Scenario

A pilot firewall policy was assigned to All Devices. Assignment audit identifies it before enforcement rollout.

Validation and troubleshooting

- Display names are not stable IDs.
- Resolve groups and deleted objects.
- Check include/exclude and filters together.

Interview-ready explanation

The safest automation is first a complete assignment report; write automation follows only after peer review.

32. Policy Backup and Documentation

What this topic means

PowerShell can export policy metadata, settings JSON, assignments, dependencies, and timestamps for documentation and diff. This is not always a guaranteed one-click restore.

Where the work starts in Intune

149. Define supported object types.
150. Export raw response and normalized summary.
151. Store in access-controlled versioned repository.
152. Diff changes.
153. Document recreation/test procedure.

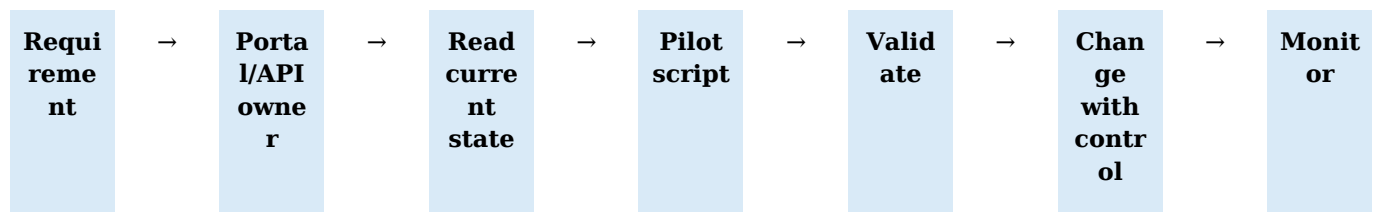
Where PowerShell is actually used

- Graph read and JSON serialization.
- Git/version control for sanitized exports.

Which PowerShell method to choose

Need	Best method
Documentation	Raw JSON + summary
Change review	Diff
Recovery	Supported restore/recreation process

Process flow



Practical PowerShell example

```

$Object=[PSCustomObject]@{
    ExportedUtc=(Get-Date).ToUniversalTime()
    ObjectType='ConfigurationPolicy'
    Id='OBJECT-ID'
    DisplayName='Example Policy'
    Assignments=@('GROUP-ID')
}
$Object | ConvertTo-Json -Depth 20 | Set-Content .\PolicyExport.json -Encoding UTF8
  
```

Real work case

Scenario

A policy is changed unexpectedly. Versioned exports show the before/after structure and assignments.

Validation and troubleshooting

- Raw exports can contain IDs and sensitive configuration.
- Test recreation in lab.
- Refresh endpoint schemas over time.

Interview-ready explanation

I call it documentation and recovery support, not universal backup, unless the exact resource has a tested restore method.

33. Graph Authentication for Daily Admin Work

What this topic means

Authentication determines who or what runs automation and with which permissions.

Where the work starts in Intune

154. Use delegated authentication for interactive admin work with MFA.
155. Use app certificate or managed identity for unattended approved automation.
156. Request minimum scopes.
157. Confirm tenant and context.
158. Disconnect and audit.

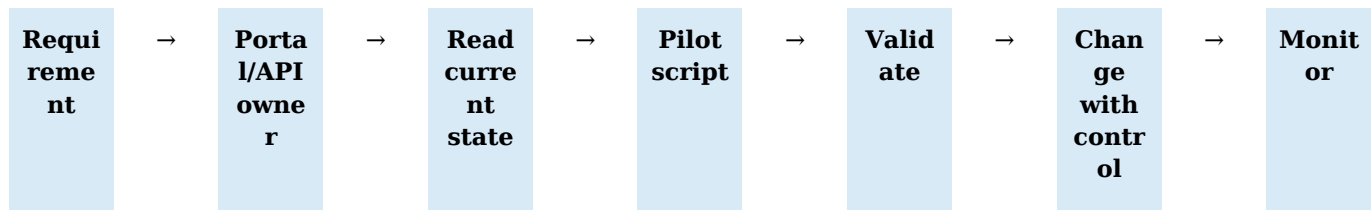
Where PowerShell is actually used

- Connect-MgGraph, Get-MgContext, Disconnect-MgGraph.
- Application auth requires Entra app, permissions, consent, credential/store, and monitoring.

Which PowerShell method to choose

Need	Best method
Interactive report	Delegated
Scheduled report	Managed identity/app certificate
Destructive change	Separate privileged workflow

Process flow



Practical PowerShell example

```

Import-Module Microsoft.Graph.Authentication
Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All'
$Context = Get-MgContext
$Context | Select-Object TenantId,Account,Scopes
# Continue only after confirming the expected tenant and read scope.
Disconnect-MgGraph
  
```

Real work case

Scenario

An admin accidentally connects to a test tenant and sees no devices. Get-MgContext immediately reveals tenant and account.

Validation and troubleshooting

- 403 means permission/consent/resource mismatch, not automatically a bad password.
- Avoid long-lived client secrets.

Interview-ready explanation

I choose auth based on interactive versus unattended use and keep permissions at the minimum required operation.

34. Graph Pagination and Large Tenant Scale

What this topic means

Graph returns paged data. A script that reads only the first page produces incomplete reports and unsafe decisions.

Where the work starts in Intune

159. Call endpoint.
160. Read value collection.
161. Follow @odata.nextLink until absent.
162. Validate count and duplicates.
163. Stream/export efficiently for large tenants.

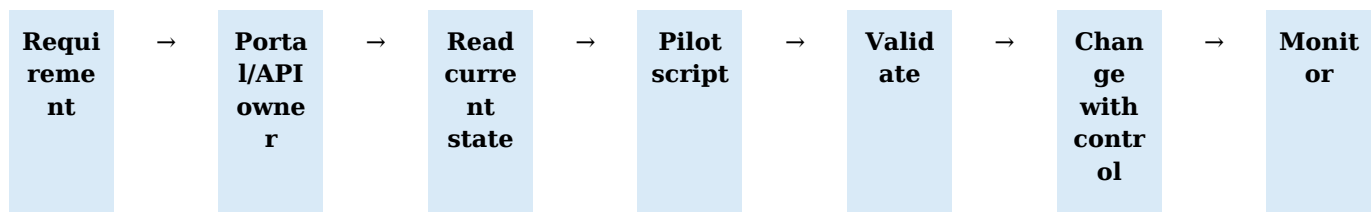
Where PowerShell is actually used

- Reusable paging function.
- Retry and Retry-After handling for throttling.

Which PowerShell method to choose

Need	Best method
Small test	Confirm schema
Full tenant	Paging and selected properties
Scheduled run	Checkpoint, retry, alerting

Process flow



Practical PowerShell example

```

function Invoke-GraphPagedGet {
    [CmdletBinding()]
    param([Parameter(Mandatory)][string]$Uri)
    $All = [System.Collections.Generic.List[object]]::new()
    do {
        $Response = Invoke-MgGraphRequest -Method GET -Uri $Uri
        foreach($Item in @($Response.value)){ $All.Add($Item) }
        $Uri = $Response.'@odata.nextLink'
    } while($Uri)
    return $All
}
  
```

Real work case

Scenario

A 5,000-device tenant report returns only the first page and falsely suggests devices are missing. Paging fixes completeness.

Validation and troubleshooting

- Use server-side select/filter when supported.
- Log request ID and nextLink failures.
- Avoid unbounded parallel requests.

Interview-ready explanation

Pagination is mandatory for trustworthy tenant automation; I validate totals and do not assume a single response is complete.

35. Graph Throttling, Retry, and Reliability

What this topic means

Graph can return 429 or transient server errors. Production scripts must pause and retry in a bounded, observable way.

Where the work starts in Intune

164. Check status code.
165. Honor Retry-After.
166. Apply exponential backoff with maximum retries.
167. Log request ID and endpoint.
168. Fail clearly after limit.

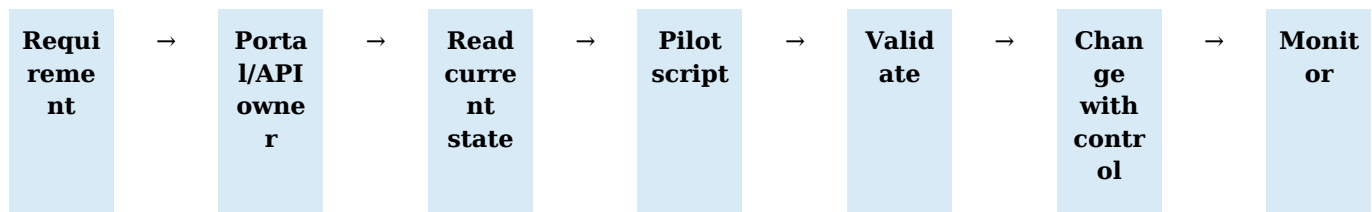
Where PowerShell is actually used

- Invoke-MgGraphRequest inside a retry wrapper.
- Do not repeat non-idempotent writes blindly.

Which PowerShell method to choose

Need	Best method
GET report	Safe bounded retry
POST/PATCH	Check idempotency and operation outcome before retry

Process flow



Practical PowerShell example

```

function Invoke-WithRetry {
    param([scriptblock]$Operation,[int]$MaxRetries=5)
    for($Attempt=0;$Attempt -le $MaxRetries;$Attempt++){
        try { return & $Operation }
        catch {
            if($Attempt -eq $MaxRetries){ throw }
            $Delay=[math]::Min([math]::Pow(2,$Attempt),60)
            Start-Sleep -Seconds $Delay
        }
    }
}

```

Real work case

Scenario

A scheduled report intermittently fails. The wrapper retries transient failures and alerts only after limit.

Validation and troubleshooting

- Honor server guidance when available.
- Do not hide permanent 400/403 errors with retries.

Interview-ready explanation

Reliability means bounded retry, logging, and idempotency, not infinite loops.

36. SCCM PowerShell Site Connection

What this topic means

Configuration Manager uses the ConfigurationManager module and site drive.

Where the work starts in Intune

169. Install/use console on supported admin workstation.
170. Import ConfigurationManager module.
171. Change to site drive.
172. Run read-only Get-CMSite and queries.
173. Use pilot collections for changes.

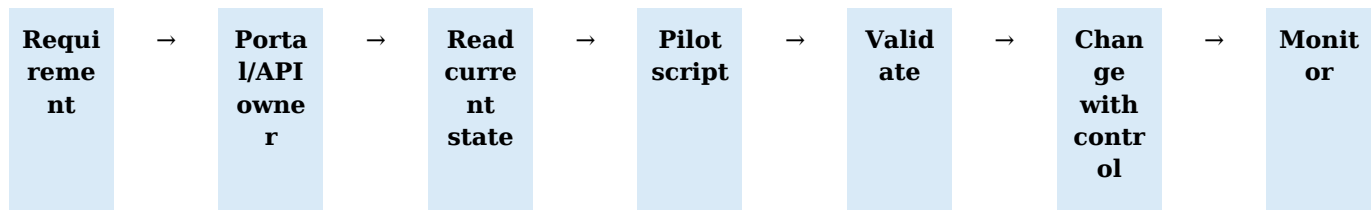
Where PowerShell is actually used

- ConfigurationManager cmdlets.
- Windows PowerShell/PowerShell version support depends on installed module and current branch.

Which PowerShell method to choose

Need	Best method
Site operations	ConfigurationManager module
Intune operations	Graph/Intune
Co-management	Both authorities with workload ownership

Process flow



Practical PowerShell example

```

Import-Module "$($env:SMS_ADMIN_UI_PATH)\..\ConfigurationManager.psd1"
$SiteCode='ABC'
Set-Location "$SiteCode\"
Get-CMSite
Get-CMCollection | Select-Object -First 10 Name,CollectionType,MemberCount
  
```

Real work case

Scenario

A cmdlet is recognized but fails outside the site drive. The admin imports the module and switches to ABC:.

Validation and troubleshooting

- Confirm console/module version.
- Check site-drive provider and permissions.

Interview-ready explanation

SCCM PowerShell is not Graph; it operates through the ConfigurationManager module and site context.

37. SCCM Collections and Device Audit

What this topic means

Collections control application, update, configuration, and task-sequence targeting.

Where the work starts in Intune

174. Query collection rules and membership.
175. Compare expected pilot scope.
176. Export direct/query-rule information.
177. Review limiting collection.
178. Change membership only through approved process.

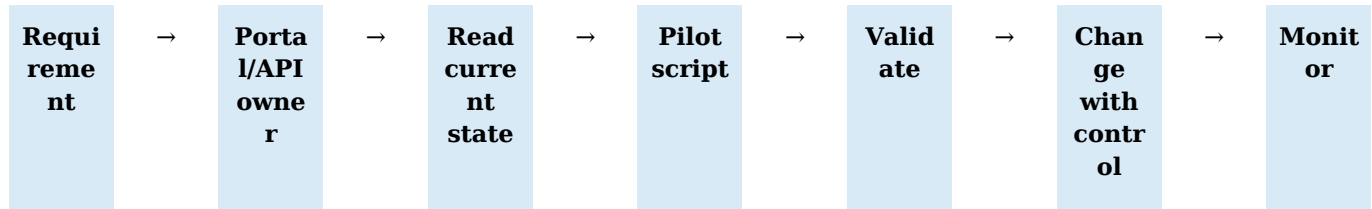
Where PowerShell is actually used

- Get-CMCollection, Get-CMDevice, rule cmdlets.

Which PowerShell method to choose

Need	Best method
Audit	Read-only SCCM cmdlets
Membership change	Controlled cmdlet/console with review

Process flow



Practical PowerShell example

```

Get-CMDevice -CollectionName 'Pilot Devices' |
Select-Object Name,ClientType,ClientVersion,LastActiveTime |
Export-Csv 'C:\Temp\PilotDevices.csv' -NoTypeInfo
  
```

Real work case

Scenario

A production deployment reaches unexpected devices because a query rule is broad. Collection audit finds the limiting collection and rule.

Validation and troubleshooting

- Collection name can be duplicated by process; use IDs where appropriate.
- Wait for membership evaluation and state updates.

Interview-ready explanation

I treat collection automation as high impact because collection membership is deployment scope.

38. SCCM Application and Content Troubleshooting

What this topic means

PowerShell can inventory application metadata and deployment targeting, while client/server logs explain failures.

Where the work starts in Intune

179. Confirm application, deployment type, detection, requirements, dependencies.
180. Confirm content distributed to DP.
181. Confirm client boundary and content location.
182. Review AppDiscovery/AppEnforce/CAS/ContentTransferManager logs.

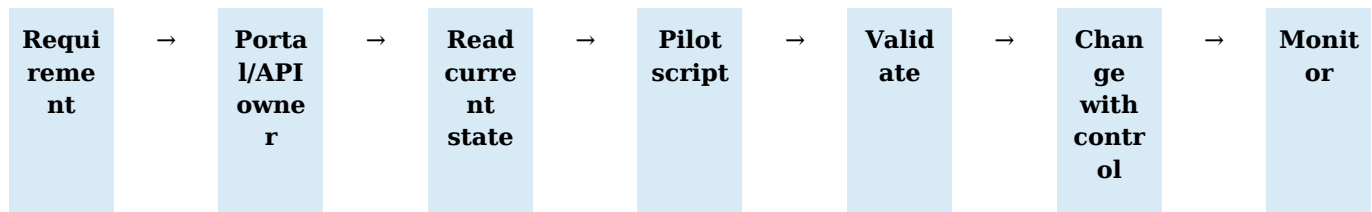
Where PowerShell is actually used

- ConfigurationManager queries for app/deployment/content status.
- Local PowerShell for file/registry/product evidence.

Which PowerShell method to choose

Need	Best method
Site metadata	SCCM cmdlets
Client execution	Local logs/PowerShell
Distribution	DP status and server logs

Process flow



Practical PowerShell example

```
Get-CMApplication | Select-Object LocalizedDisplayName, SoftwareVersion, DateLastModified
# Run in site drive; use read-only inventory before any deployment change.
```

Real work case

Scenario

An application installs manually but SCCM reports failed. Detection method and AppDiscovery.log reveal the mismatch.

Validation and troubleshooting

- Do not redistribute content before finding root cause.
- Correlate policy, content, execution, detection, and state message.

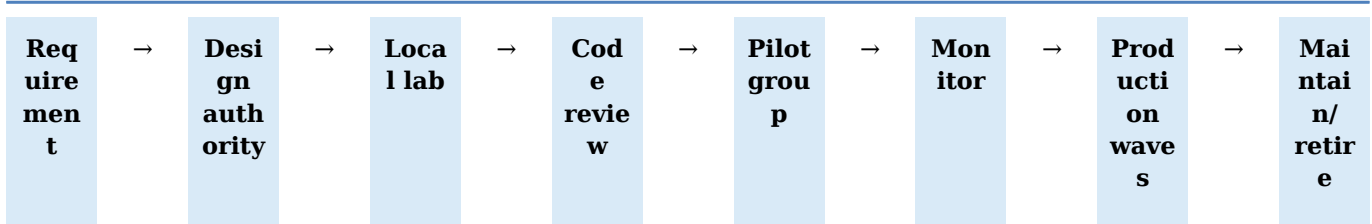
Interview-ready explanation

PowerShell shortens inventory and correlation; SCCM logs remain the root-cause evidence.

39. Practical Decision Matrix

Work request	Use PowerShell where?
Create/report tenant policy	Admin PowerShell + Microsoft Graph
Run script on Windows endpoint	Intune Platform Script
Detect and fix recurring endpoint problem	Intune Remediation
Package/install app	PowerShell wrapper + Intune Win32 app
Check local security/configuration state	Local PowerShell
Query SCCM site objects	ConfigurationManager module
Single destructive device action	Portal or tightly controlled Graph workflow
Bulk lifecycle action	Discovery report, human approval, controlled execution

40. Script Development Lifecycle



Phase	Evidence
Requirement	Owner, expected state, affected platform, risk
Design	Portal/Graph/local/remediation choice and permissions
Build	Versioned source, functions, logs, comments
Test	Compliant, noncompliant, error, rollback cases
Pilot	Limited group and acceptance criteria
Production	Change approval, monitoring, support communication
Maintain	Owner, version, expiry/review date, incident history

41. Troubleshooting a Script That Works Manually but Fails in Intune

183. Confirm the exact script version uploaded.
184. Confirm Intune assignment and device/user scope.
185. Confirm IME installation, service, sync, and logs.
186. Reproduce under SYSTEM or user context matching the portal setting.
187. Confirm 32-bit or 64-bit PowerShell host.
188. Check current working directory and use \$PSScriptRoot.
189. Remove assumptions about mapped drives, HKCU, interactive prompts, and UI.
190. Check execution policy/signature requirement and security product.
191. Check network/proxy/TLS under SYSTEM.
192. Check exit code, output, timeout, and error stream.
193. Verify the script did not succeed while validation/detection failed.

```
# Context evidence
whoami
[Environment]::Is64BitProcess
$PSScriptRoot
$PWD.Path
$env:USERPROFILE
Get-Service IntuneManagementExtension -ErrorAction SilentlyContinue
```

42. Reusable Production Script Header

```
<#  
.SYNOPSIS  
    Short purpose.  
.DESCRIPTION  
    Business owner, technical owner, supported platforms, execution context.  
.PARAMETER  
    Document every parameter and allowed value.  
.OUTPUTS  
    PSObject or documented text/exit codes.  
.NOTES  
    Version, change ticket, permissions, rollback, data classification.  
#>  
[CmdletBinding(SupportsShouldProcess)]  
param()  
Set-StrictMode -Version Latest  
$ErrorActionPreference='Stop'  
$ScriptVersion='1.0.0'  
# Initialize privacy-safe logging and validate prerequisites here.
```

43. 90-Day Learning Plan

Days	Focus	Output
1-15	PowerShell syntax, objects, pipeline, functions	Ten local read-only scripts
16-30	Errors, logs, registry, services, CIM, event logs	Endpoint diagnostic package
31-45	Platform Scripts and IME troubleshooting	Pilot script in lab group
46-60	Remediations and Pester tests	Detection/remediation pair
61-75	Graph auth, paging, Intune device/app/policy reports	Read-only tenant inventory
76-90	SCCM module, capstone, interview	Portfolio and mock interview

Daily practice

Read one concept, type every command manually in a lab, explain the object output aloud, change one parameter, break it deliberately, read the error, fix it, and write the lesson learned.

44. Interview Questions

Where exactly is PowerShell used in Intune?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How is a Platform Script different from Graph automation?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How is a Remediation created and how do exit codes work?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you create a policy with PowerShell?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you validate a configuration policy locally?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you report all managed devices?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you handle Graph pagination and throttling?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you secure unattended automation?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you troubleshoot IME?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you automate application packaging?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you use PowerShell with SCCM?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

How do you prevent accidental bulk actions?

Use this answer order: business requirement, portal or management authority, PowerShell location, authentication/context, script/process steps, safety, validation, troubleshooting, and a real work example.

45. Official Reference Map

- [Microsoft Learn: Intune devices and apps API overview.](#)
- [Microsoft Learn: Working with Intune in Microsoft Graph.](#)
- [Microsoft Learn: Use Remediations to detect and fix support issues.](#)
- [Microsoft Learn: PowerShell scripts for Remediations.](#)
- [Microsoft Learn: Configuration Manager PowerShell cmdlets.](#)
- [Microsoft GitHub: Microsoft Graph Intune samples.](#)

Maintenance

Before production use, verify current Graph resource, API version, permissions, Intune portal path, script requirements, licensing, and platform support. Use beta resources only when the organizational design accepts the change risk.